Informed search Strategies

Introduction

- Informed search is a type of search algorithm in artificial intelligence that uses **additional information or heuristics to make more accurate decisions about which paths to explore first.**
- These heuristics provide **estimates of how close a given state is to the goal**, guiding the search toward more promising solutions.
- Informed search is particularly useful in solving complex problems efficiently, as it can significantly reduce the search space and improve the speed of finding solutions.

Best-first search

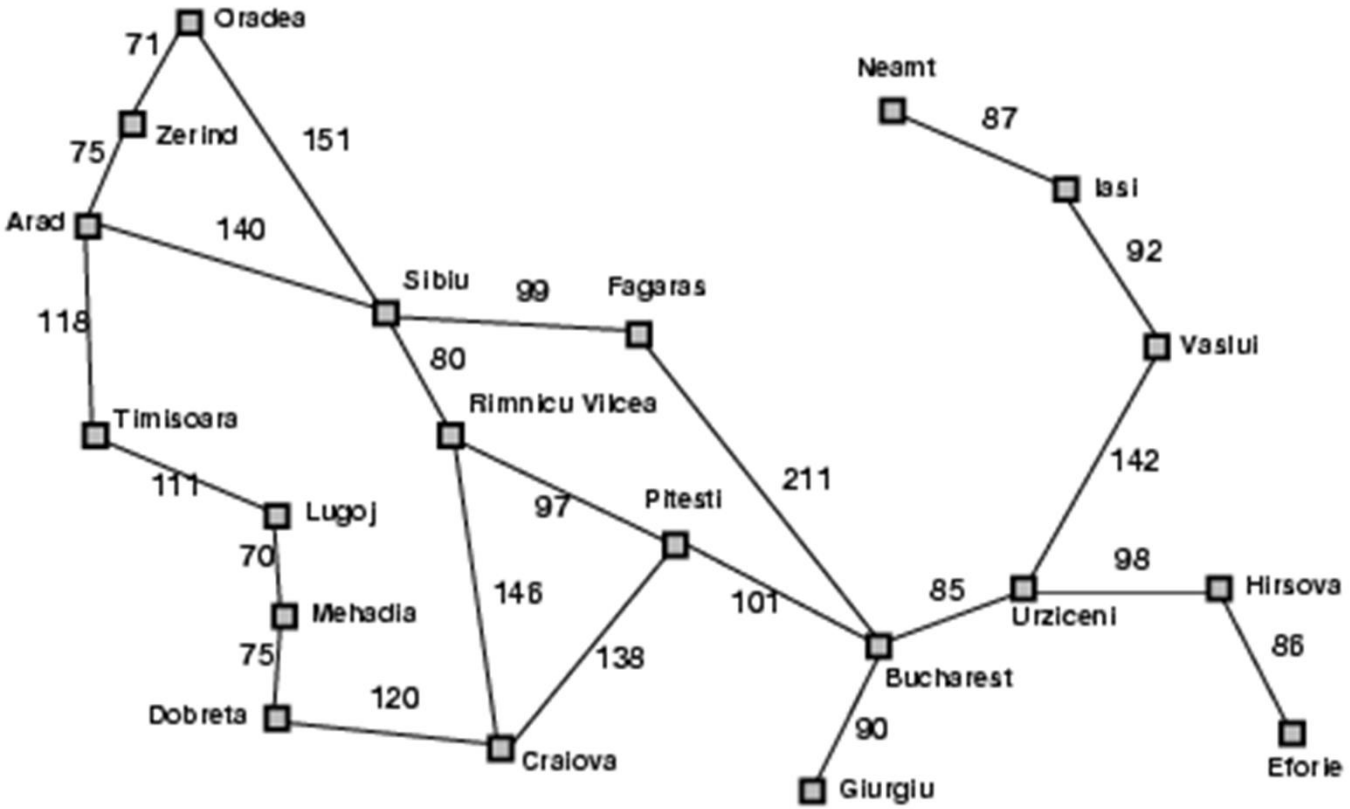
- Idea: use an evaluation function *f(n)* for each node
  - estimate of "desirability"
  - Expand most desirable unexpanded node

- Implementation:

  Order the nodes in fringe(Data structure) in decreasing order of desirability

- Special cases:
  - greedy best-first search
  - A^* search

Romania with step costs in km



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search

- Evaluation function *f(n) = h(n)* (heuristic) = estimate of cost from current node(n) to *goal*
- e.g., $h_{SLD}(n)$ = straight-line distance from *n* to Bucharest
- Greedy best-first search expands the node that appears to be closest to goal.

Greedy Best-First Search Algorithm

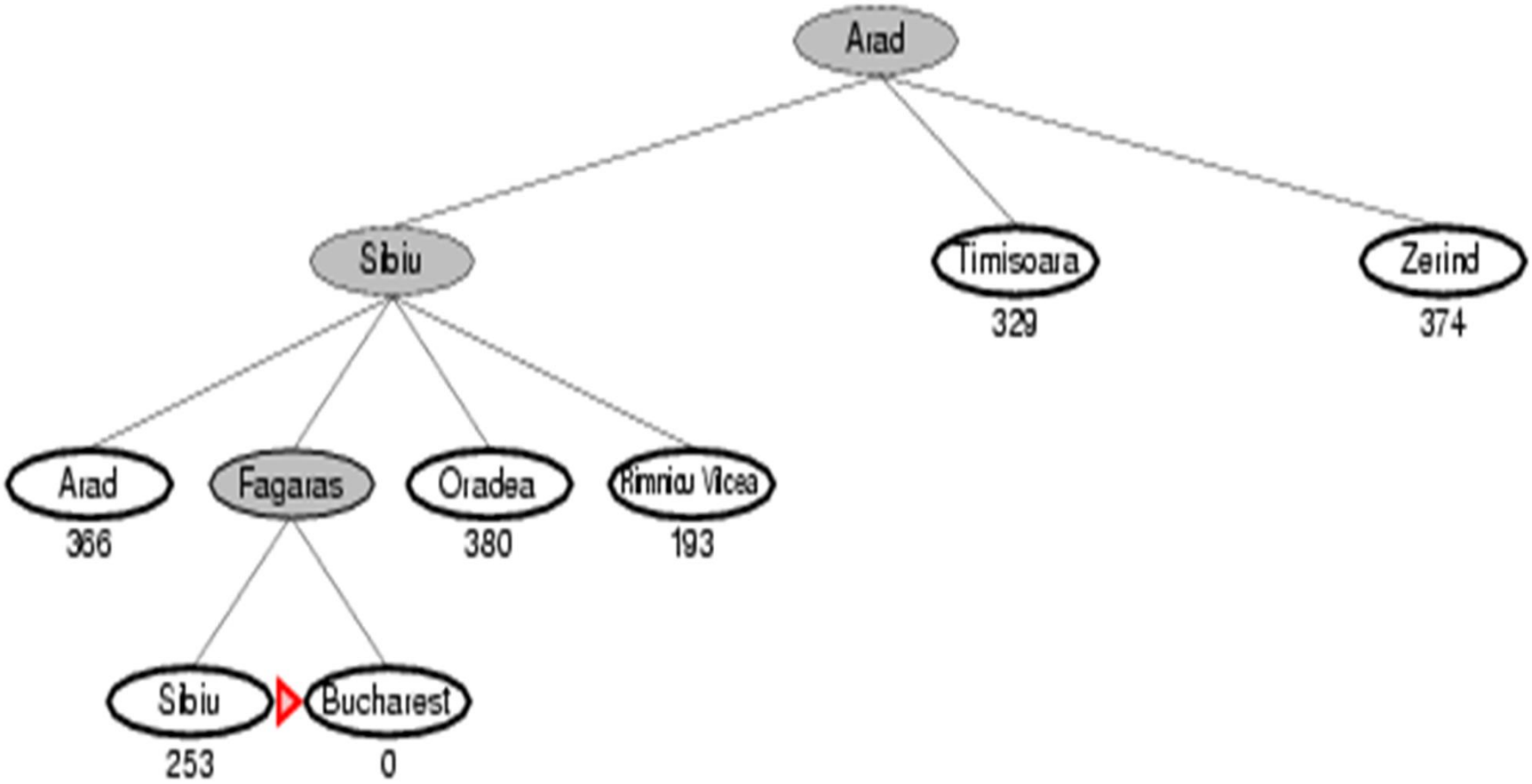
1. **Initialization:**
   - Initialize the priority queue PQ with the start node S and heuristic cost $h(S)$.
   - Initialize the explored set $\mathrm{Explored}$ as empty.
   - Priority Queue: $\mathrm{PQ} = \{(h(S), S)\}$
2. **While PQ is not empty:**
   1. Remove the node n with the lowest heuristic cost $h(n)$ from PQ.
   2. If n is the goal node, return the path.
   3. If n is not in $\mathrm{Explored}$, add n to $\mathrm{Explored}$.
   4. For each neighbor m of n:
      - Calculate the heuristic cost $h(m)$.
      - If m is not in $\mathrm{Explored}$ or PQ, add m to PQ with $h(m)$.

Greedy best-first search example

Greedy best-first search example

Greedy best-first search example

Greedy best-first search example



Properties of greedy best-first search

- Complete? No – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →
-
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
-
- Space? $O(b^m)$ -- keeps all nodes in memory
-
- Optimal? No
-

A* search

- Idea: avoid expanding paths that are already expensive or combine best features of UCS and GBFS

Evaluation function $f(n) = g(n) + h(n)$

$g(n)$ = cost so far to reach n

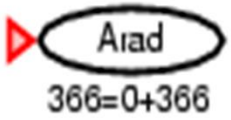
$h(n)$ = estimated cost from n to goal

$f(n)$ = estimated total cost of path through n to goal

A* Search Algorithm

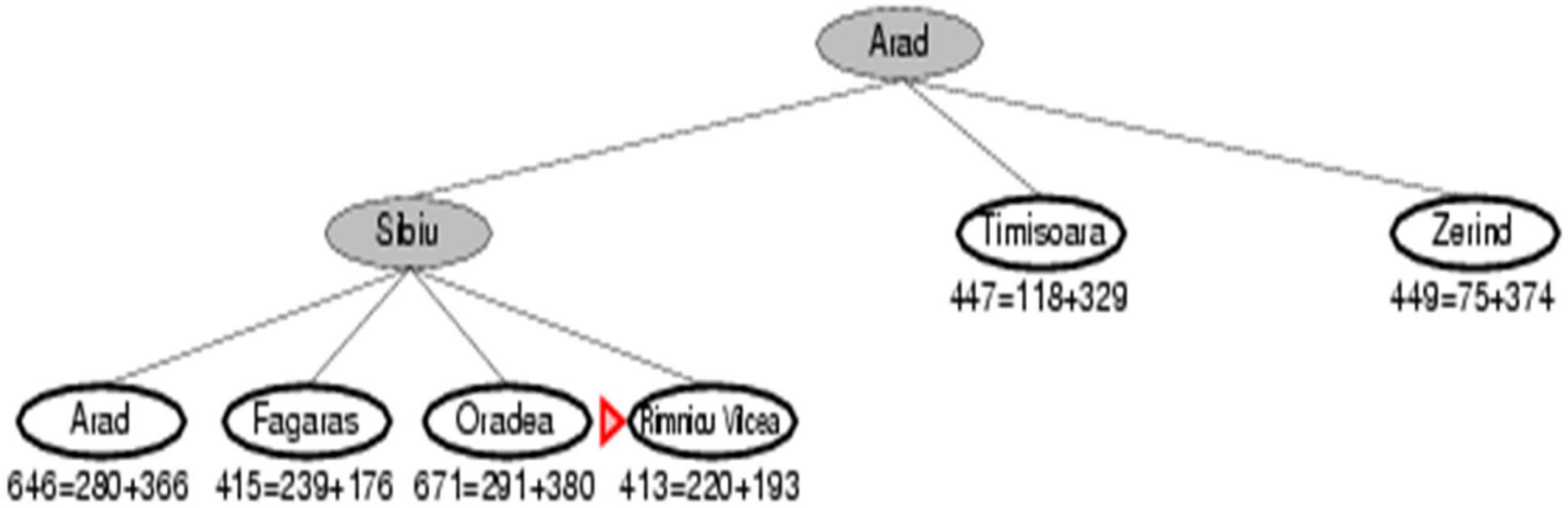
1. **Initialization:**
   - Initialize the priority queue PQ with the start node S, path cost $g(S) = 0$, and heuristic cost $h(S)$.
   - Initialize the explored set $\mathrm{Explored}$ as empty.
   - Priority Queue: $\mathrm{PQ} = \{(f(S), S)\}$ where $f(S) = g(S) + h(S)$
2. **While PQ is not empty:**
   1. Remove the node n with the lowest $f(n)$ from PQ where $f(n) = g(n) + h(n)$.
   2. If n is the goal node, return the path and cost.
   3. If n is not in $\mathrm{Explored}$, add n to $\mathrm{Explored}$.
   4. For each neighbor m of n:
      - Calculate the new path cost $g(m) = g(n) + \mathrm{cost}(n, m)$.
      - Calculate $f(m) = g(m) + h(m)$.
      - If m is not in $\mathrm{Explored}$ or PQ, add m to PQ with $f(m)$.
      - If m is in PQ with a higher $f(m)$, update m in PQ with the new lower $f(m)$.

A* search example

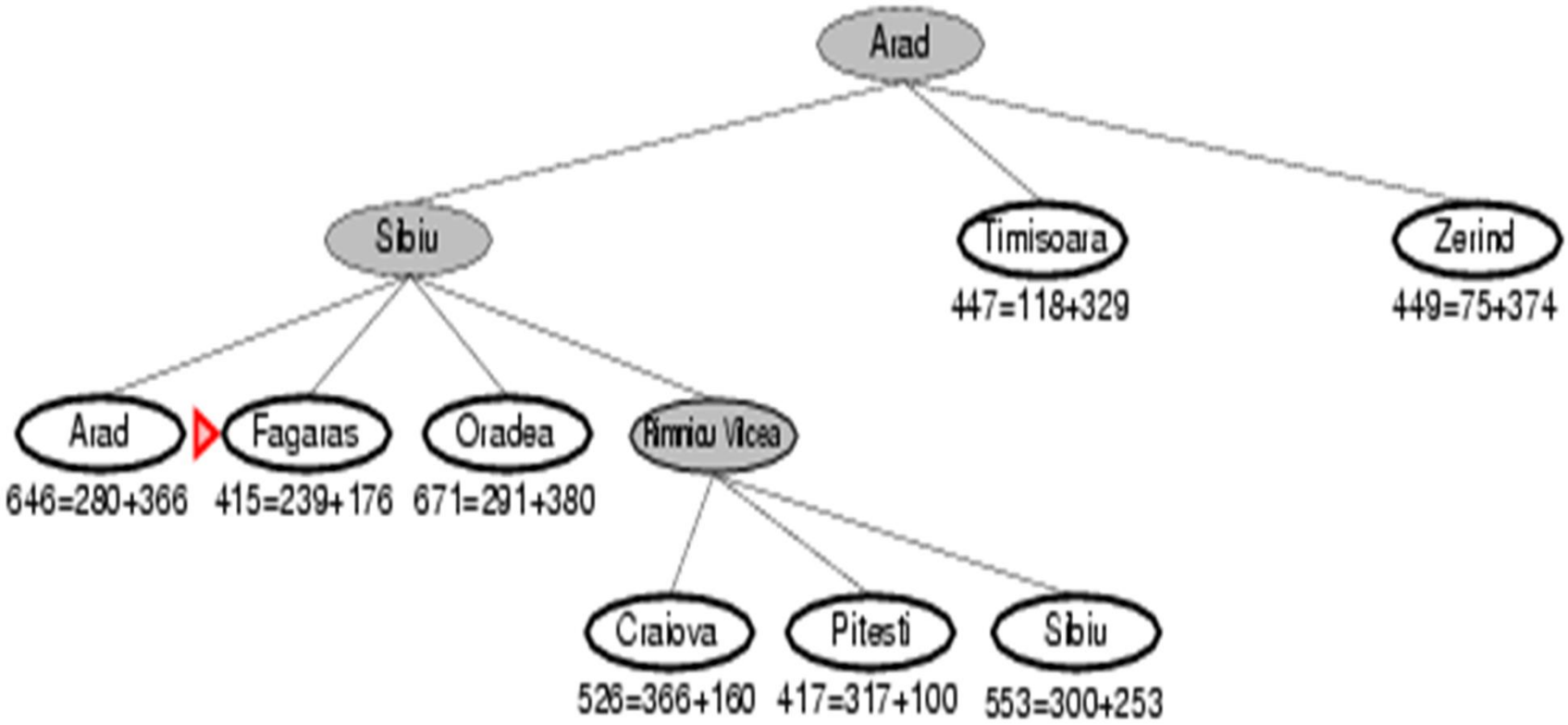


A* search example

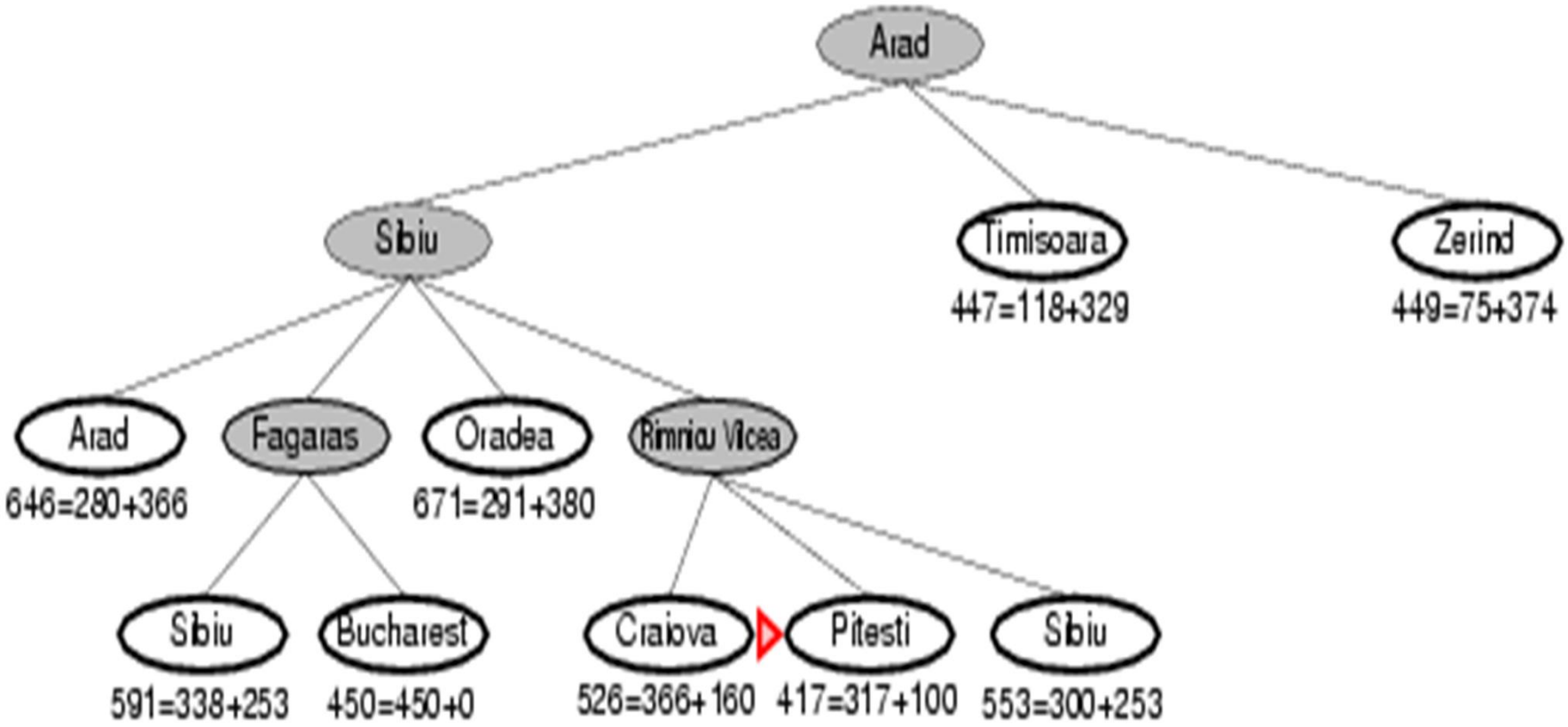
A* search example



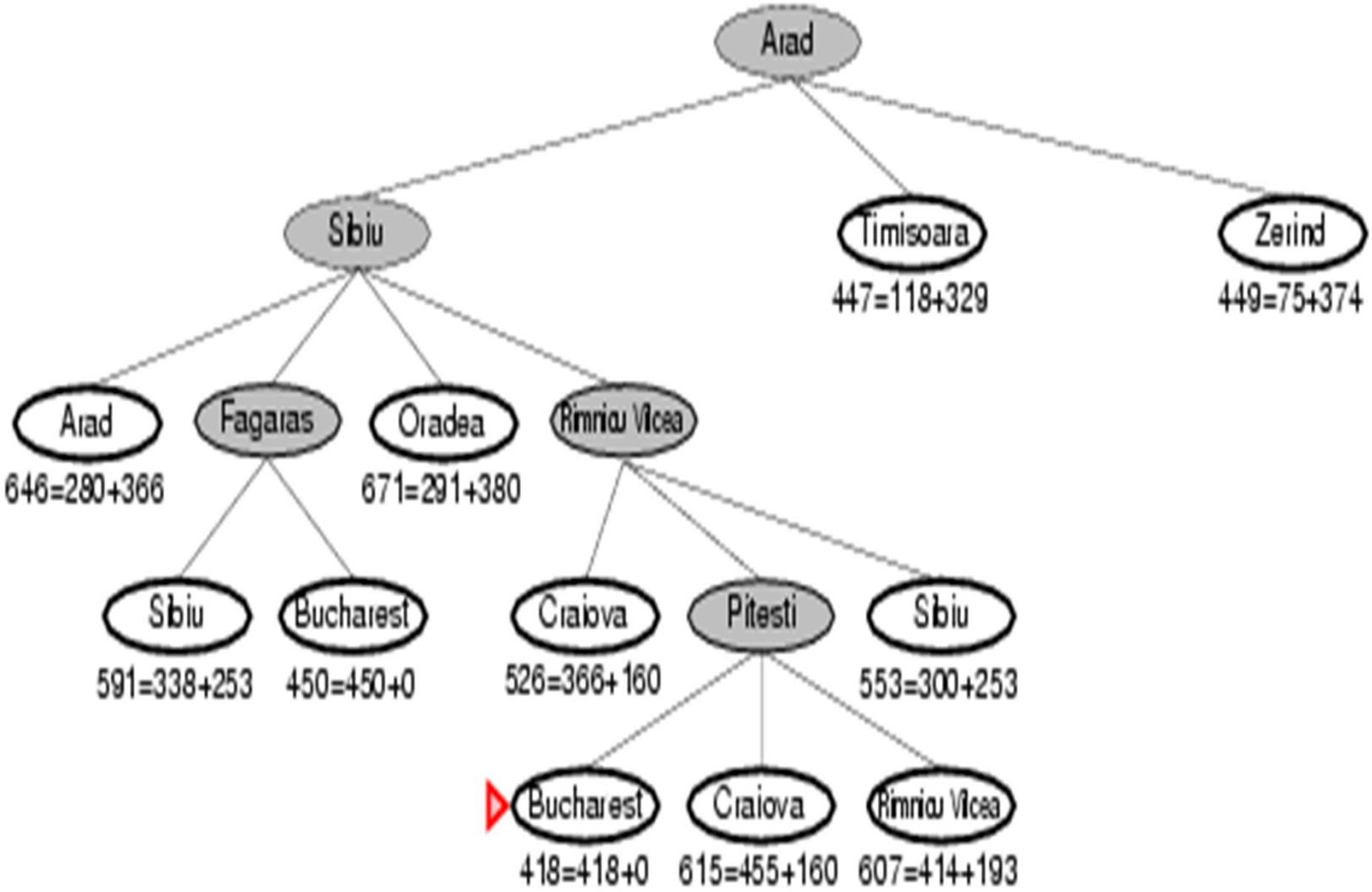
A* search example



A* search example



A* search example



Summary of Key Points

- **UCS**: Focuses on the path cost $g(n)$ and re-adds nodes if a cheaper path is found before expansion.
- **Greedy Best-First Search**: Focuses on the heuristic $h(n)$ and typically does not re-add nodes based on path cost since it doesn't consider $g(n)$.
- **A***: Combines path cost $g(n)$ and heuristic $h(n)$ into $f(n) = g(n) + h(n)$ and re-adds nodes if a path with a lower $f(n)$ is found before expansion.

Admissible heuristics

- A heuristic *h(n)* is admissible if for every node *n*,
  $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from *n*.

- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
-
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- Theorem: If *h(n)* is admissible, A* using `TREE-SEARCH` is optimal